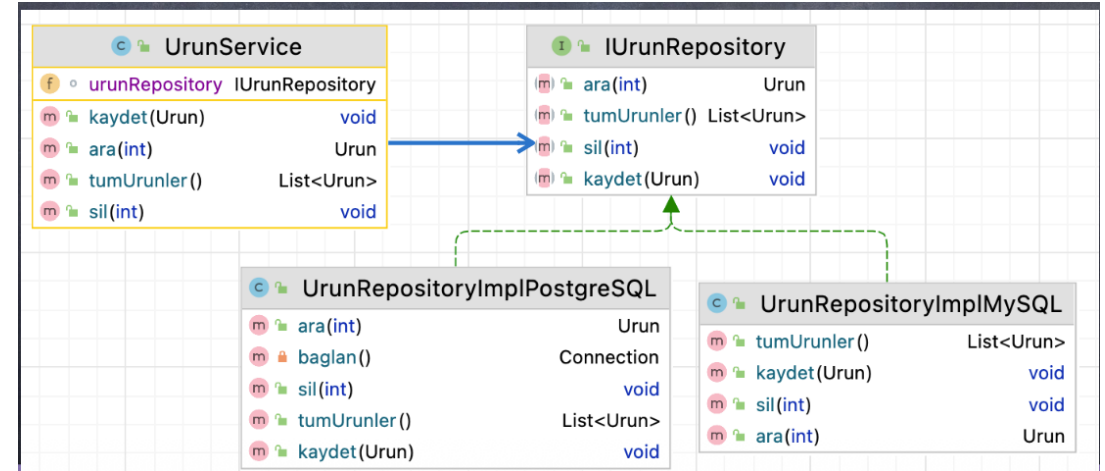


NDAT

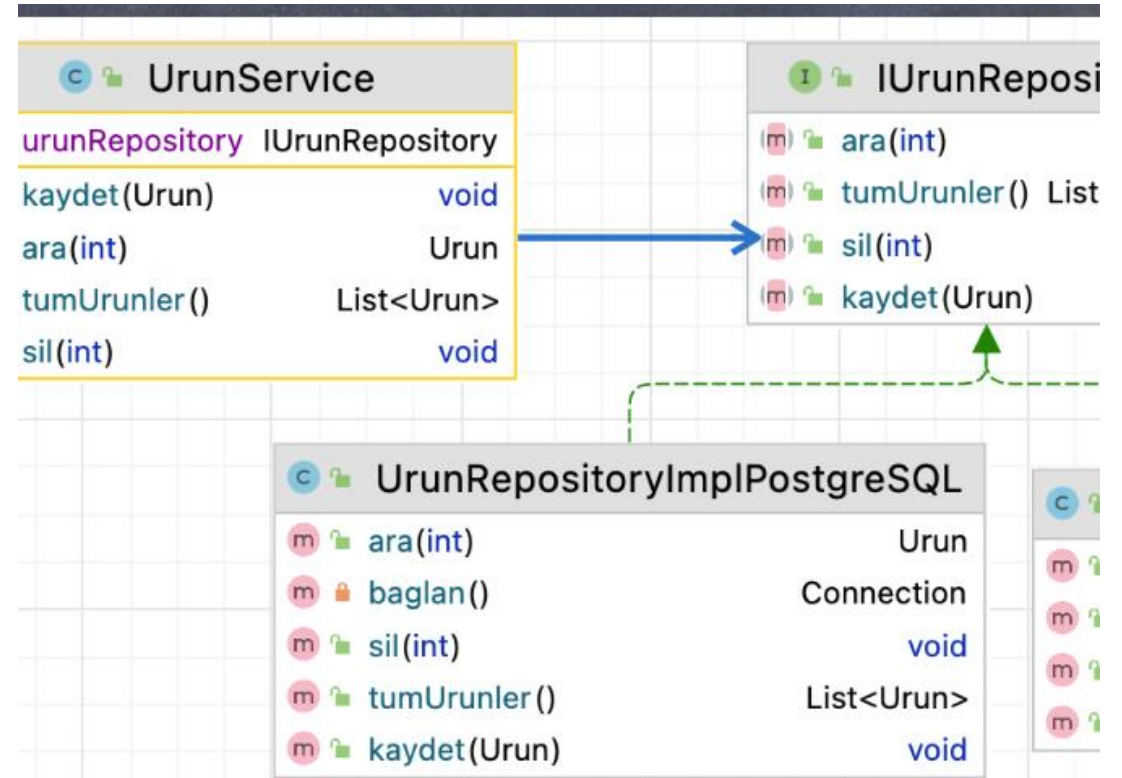
Sorular

1. Veritabanında bulunan ürünlerin sayısını hesaplayan yöntemi ekleyiniz.
2. Ürün adına göre arama yapılabilmesini sağlayınız.
3. Ürün bilgilerinin güncellenebilmesini sağlayınız.
4. Urun sınıfı içerisinde stok miktarı (stokMiktari) da saklanmaktadır. Stokta bulunan ürünlerin tespitini sağlayacak yöntemi (urunMevcutMu(int)) de ekleyiniz.



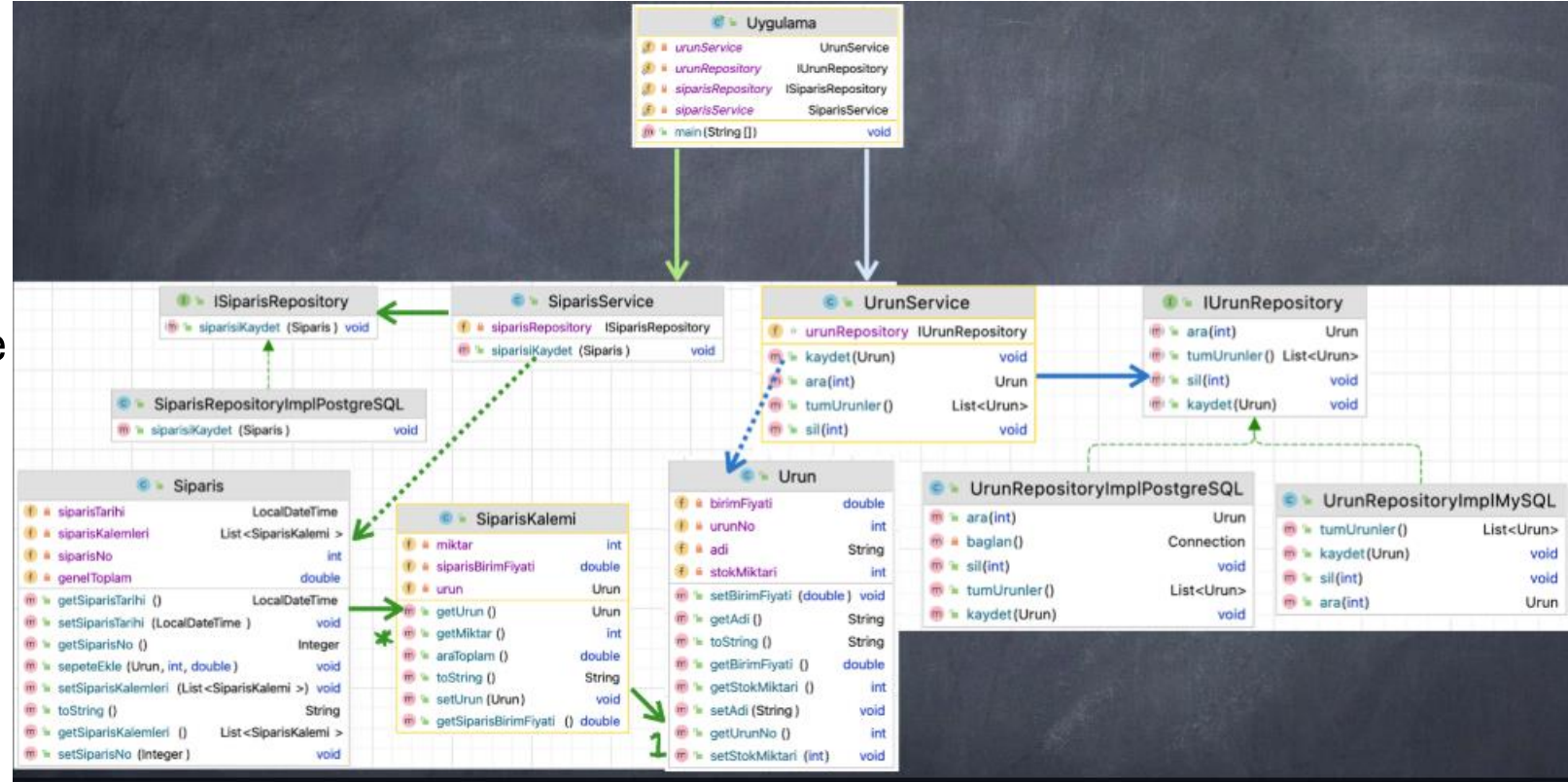
Sorular

- 5. `urunMevcutMu(int)` metodu `UrunService` içerisindeki `tumUrunler()` yöntemi içerisinde stokta bulunan ürünlerin döndürülmesini sağlamak için kulan.
- Uygulamanın MongoDB için de çalışabilmesini sağlayınız.



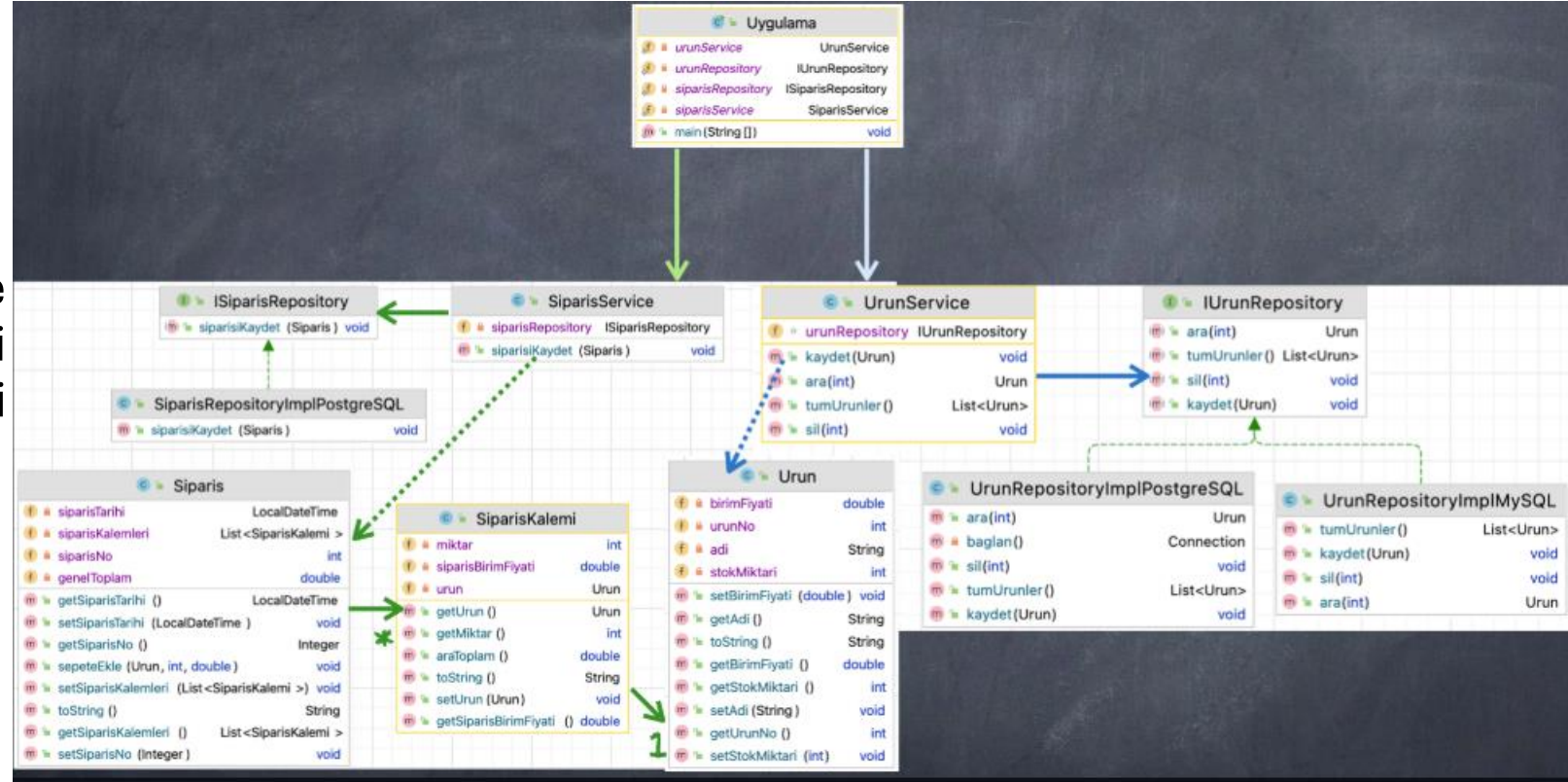
Sorular

- Sipariş veren müşteri bilgilerinin de eklenmesini sağlayınız.



Sorular

- Soru2: Mevcut uygulamada, sepete eklenecek ürünün hatalı girilmesi durumunu ele alan iyileştirmeyi yapınız.



Sorular

Soru2: Mevcut uygulamada, sepete eklenecek ürünün hatalı girilmesi durumunu ele alan iyileştirmeyi yapınız.

- Ürün varlık kontrolü yapılmalıdır.

// Uygulama.java - do-while döngüsü içi

urun = tumUrunlerinListesi.stream()

.filter(urunElementi -> urunAdi.equalsIgnoreCase(urunElementi.getAdi()))

.findAny()

.orElse(null);

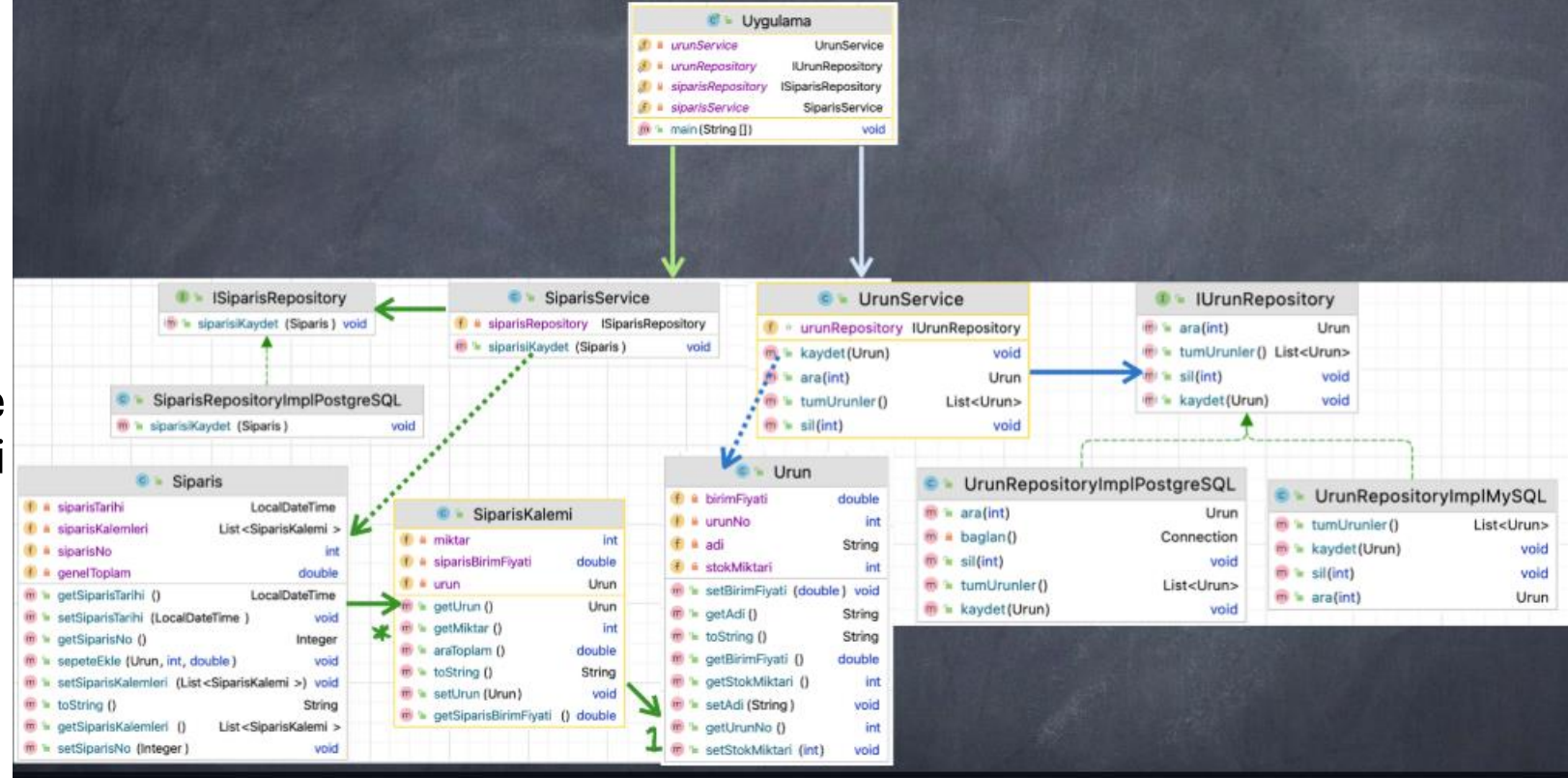
// İYİLEŞTİRME: Ürün kontrolü

if (urun == null) {

System.out.println("Hata: Girdiğiniz isimde bir ürün bulunamadı! Lütfen listeden bir ürün seçin.");

continue; // Döngünün başına döner, miktar sormaz

}



Soru2: Mevcut uygulamada, sepete eklenecek ürünün hatalı girilmesi durumunu ele alan iyileştirmeyi yapınız.

- Stok ve Miktar Kontrolü yapılmalıdır.

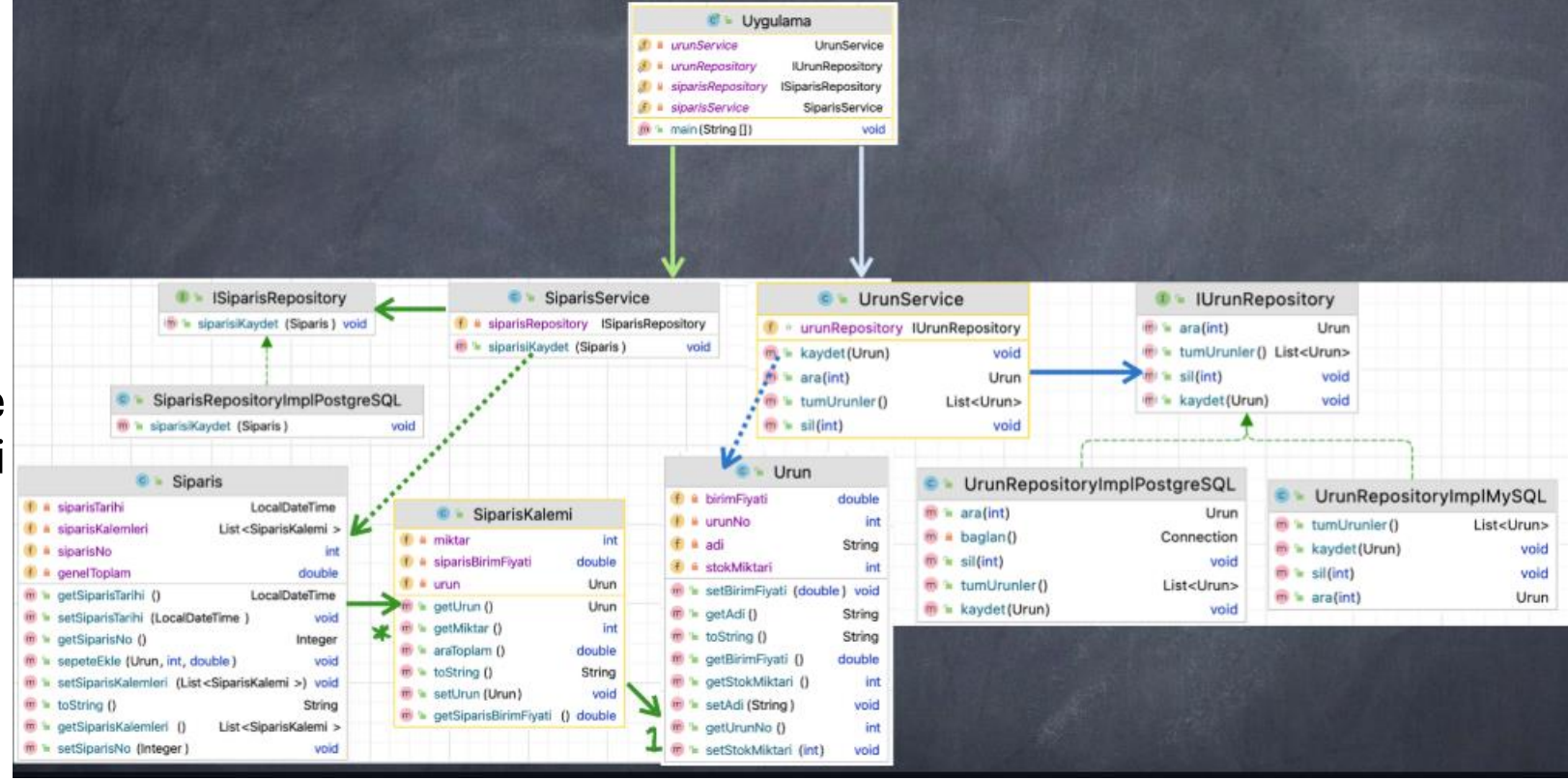
// Siparis.java içindeki metot iyileştirmesi

```
public void sepeteEkle(Urun urun, int miktar, double siparisBirimFiyati) {  
    // 1. Ürün null mı?  
    if (urun == null) {  
        throw new IllegalArgumentException("Sepete boş ürün eklenemez!");  
    }  
  
    // 2. Miktar geçerli mi?  
    if (miktar <= 0) {  
        System.out.println("Hata: Miktar 0 veya daha az olamaz.");  
        return;  
    }  
  
    // 3. Stok yeterli mi? (Urun sınıfında getStokMiktari olduğu varsayılırsa)  
    if (miktar > urun.getStokMiktari()) {  
        System.out.println("Hata: Yetersiz stok! Mevcut stok: " + urun.getStokMiktari());  
        return;  
    }  
}
```

Sorular

Soru2: Mevcut uygulamada, sepete eklenecek ürünün hatalı girilmesi durumunu ele alan iyileştirmeyi yapınız.

- Fiyat doğrulama iyileştirmesi



// Kullanıcıdan fiyat almak yerine direkt üründen çekmek daha güvenlidir:

```
System.out.println("Eklemek istediğiniz miktarı giriniz:");
miktar = giris.nextInt(); giris.nextLine();
```

// Birim fiyatı kullanıcıdan değil, sistemdeki ürün nesnesinden alıyoruz

```
double sistemdekiFiyat = urun.getBirimFiyati();
```

```
siparis.sepeteEkle(urun, miktar, sistemdekiFiyat);
```

Hibernate

- Hibernate, bir **ORM (Object-Relational Mapping)** aracıdır. Java nesneleri ile veritabanı tabloları arasındaki köprüdür.
- **Amacı:** Yazılımcıyı SQL yazma zahmetinden kurtarıp, veritabanı işlemlerini tamamen Java nesneleri üzerinden yapmasını sağlamaktır.
- **Ne Yapar?**
 - Sen Java'da bir nesne oluşturursun, o arka planda INSERT sorgusu yazar.
 - Sen findById dersin, o arka planda SELECT sorgusu yazar.
 - Veritabanı tablolarını senin Java sınıflarına bakarak otomatik oluşturur.

Spring Boot

- Spring Boot, karmaşık Java uygulamalarını **hızlıca ayağa kaldırmak** için kullanılan bir framework'tür.
- **Amacı:** Geliştiricinin ayar (konfigürasyon) dosyalarıyla boğuşmasını engellemek ve "çalıştırılabilir" bir uygulama sunmaktır.
- **Ne Yapar?** * Sunucuyu (Tomcat) kendi içinde hazır getirir.
 - Bağımlılıkları (Dependency) yönetir.
 - Uygulamanın tüm katmanlarını (Service, Repository, Controller) bir arada tutar

Kıyaslama Tablosu

Özellik	Spring Boot	Hibernate
Kapsam	Uygulamanın tamamını kapsar (Bütünsel).	Sadece veritabanı katmanına odaklanır.
Görevi	Uygulamayı başlatır, servisleri yönetir.	Java nesnelerini veritabanına kaydeder.
Bağımlılık	Hibernate'i içinde barındırabilir.	Spring olmadan da tek başına kullanılabilir.
SQL Bilgisi	SQL ile doğrudan ilgilenmez.	Senin yerine SQL üretir.

JPA (Java Persistence API) Nedir?

- JPA, Java dünyasında veritabanı işlemleri için konulmuş bir **standartlar setidir**.
- **Sadece bir "Arayüz"dür (Interface):** JPA kendi başına hiçbir iş yapmaz. Sadece "Bir veritabanı sınıfı nasıl olmalı?", "İlişkiler nasıl tanımlanmalı?" (Örn: @Entity, @Id, @OneToMany) sorularının cevaplarını ve kurallarını belirler.
- **Amacı:** Yazılımcının kodunu veritabanı kütüphanesinden bağımsız hale getirmektir. Bugün Hibernate kullanırsın, yarın başka bir kütüphaneye (EclipseLink gibi) geçersen kodunu değiştirmek zorunda kalmazsın.

JPA (Java Persistence API) Nedir?

- JPA, Java dünyasında veritabanı işlemleri için konulmuş bir **standartlar setidir**.
- **Sadece bir "Arayüz"dür (Interface):** JPA kendi başına hiçbir iş yapmaz. Sadece "Bir veritabanı sınıfı nasıl olmalı?", "İlişkiler nasıl tanımlanmalı?" (Örn: @Entity, @Id, @OneToMany) sorularının cevaplarını ve kurallarını belirler.
- **Amacı:** Yazılımcının kodunu veritabanı kütüphanesinden bağımsız hale getirmektir. Bugün Hibernate kullanırsın, yarın başka bir kütüphaneye (EclipseLink gibi) geçersen kodunu değiştirmek zorunda kalmazsın.

Özet

- Spring Boot:** Projeyi yönetir ve kütüphaneleri bir araya getirir.
- Spring Data JPA:** Veritabanı işlerini çok kolay bir arayüzle (Repository) bize sunar.
- JPA:** Standartları belirler (Anatasyonları sağlar: @Entity gibi).
- Hibernate:** Arka planda gerçek SQL'i üreten ve PostgreSQL'e gönderen asıl motordur.

Entity

- **Nerede:** Siparis.java, Urun.java ve SiparisKalemi.java dosyalarında.
- Bu sınıflar bizim veritabanındaki tablolarımızın aynasıdır.
- Sınıfın üstüne @Entity yazdığında Spring'e şunu dersin: *"Bu sadece bir Java sınıfı değil, bunu git veritabanında bir tablo yap."*
- @Table(name = "urun") diyerek tablonun adını koyarsın.
- **İlişki (Relationship):** Siparis içinde @OneToMany yazdık. Bu sayede bir sipariş kaydedildiğinde, Spring onun içindeki listeye bakıp "Ha, bu siparişin kalemleri de varmış, onları da kaydedeyim" diyor.

Entity

- **Nerede:** Siparis.java, Urun.java ve SiparisKalemi.java dosyalarında.
- Bu sınıflar bizim veritabanındaki tablolarımızın aynasıdır.
- Sınıfın üstüne @Entity yazdığında Spring'e şunu dersin: *"Bu sadece bir Java sınıfı değil, bunu git veritabanında bir tablo yap."*
- @Table(name = "urun") diyerek tablonun adını koyarsın.
- **İlişki (Relationship):** Siparis içinde @OneToMany yazdık. Bu sayede bir sipariş kaydedildiğinde, Spring onun içindeki listeye bakıp "Ha, bu siparişin kalemleri de varmış, onları da kaydedeyim" diyor.

Entity

- **Nerede:** Siparis.java, Urun.java ve SiparisKalemi.java dosyalarında.
- Bu sınıflar bizim veritabanındaki tablolarımızın aynasıdır.
- Sınıfın üstüne @Entity yazdığında Spring'e şunu dersin: *"Bu sadece bir Java sınıfı değil, bunu git veritabanında bir tablo yap."*
- @Table(name = "urun") diyerek tablonun adını koyarsın.
- **İlişki (Relationship):** Siparis içinde @OneToMany yazdık. Bu sayede bir sipariş kaydedildiğinde, Spring onun içindeki listeye bakıp "Ha, bu siparişin kalemleri de varmış, onları da kaydedeyim" diyor.

Entity ve Normal Sınıf

- Farkı nedir?

Relationship

- Many-To-One:

@ManyToOne

@JoinColumn(name = "siparis_id")

private Siparis siparis;

- One-To-Many:

@OneToMany(mappedBy = "siparis", cascade = CascadeType.ALL)

private List<SiparisKalemi> siparisKalemleri = new ArrayList<>();

Relationship

- Many-To-Many:

@ManyToMany

@JoinTable(

name = "ogrenci_dersleri",

joinColumns = @JoinColumn(name = "ogrenci_id"),

inverseJoinColumns = @JoinColumn(name = "ders_id")

)

private List<Ders> dersler;

JpaRepository

- **Nerede:** IUr repository.java ve ISiparisRepository.java dosyalarında.
- extends JpaRepository: Java'nın dev kütüphanesi senin yerine tüm SQL kodlarını hazırlıyor.
- INSERT INTO... Yerine save()

JpaRepository Metotları

- save(S)**: Eğer nesne veritabanında yoksa INSERT (Ekleme), varsa UPDATE (Güncelleme) yapar.
- findById(ID)**: Verilen ID'ye göre tek bir satır getirir (SELECT * FROM ... WHERE id = ?).
- findAll()**: Tablodaki tüm verileri bir liste (List) olarak döner.
- deleteById(ID)**: Verilen ID'ye sahip satırı siler.
- count()**: Tabloda kaç kayıt olduğunu sayar.

ISiparisRepository boş mu? Proxy Tasarım Kalıbı

- ISiparisRepository fiziksel olarak boş olsa da, aslında JpaRepository sınıfından çok büyük bir miras devralıyor.
- Sen extends JpaRepository<Siparis, Integer> yazdığın an, o "boş" interface'in içine otomatik olarak şu metotların tanımları yükleniyor:
 - save() (Kaydetme/Güncelleme)
 - findById() (ID ile bulma)
 - findAll() (Hepsini getirme)
 - delete() (Silme)

ISiparisRepository boş mu? Proxy Tasarım Kalıbı

- ISiparisRepository fiziksel olarak boş olsa da, aslında JpaRepository sınıfından çok büyük bir miras devralıyor.
- Sen extends JpaRepository<Siparis, Integer> yazdığın an, o "boş" interface'in içine otomatik olarak şu metotların tanımları yükleniyor:
 - save() (Kaydetme/Güncelleme)
 - findById() (ID ile bulma)
 - findAll() (Hepsini getirme)
 - delete() (Silme)

Service

- **Nerede:** UrunService.java ve SiparisServisi.java dosyalarında.
- @Service etiketiyle işaretlenir.
- **Bağımlılık Enjeksiyonu (Dependency Injection):** Servis sınıfının içinde `private ISiparisRepository siparisRepository;` yazdık. Spring, uygulama çalışırken gidip o interface'in çalışan bir kopyasını (Bean) oluşturup buraya "enjekte" ediyor. Yani sen `new` demiyorsun, Spring senin yerine hallediyor.
- **@Transactional:** Bu çok kritik. Siparişi kaydederken elektrik kesilirse veya bir hata olursa, yarım yamalak kayıt yapılmasın diye her şeyi geri alır. "Ya hep ya hiç" kuralıdır.

Komuta Merkezi (Main / Application)

- **Nerede:** Main.java dosyasında.

- `@SpringBootApplication` : Spring motorunu çalıştıran ana anahtardır.
- `CommandLineRunner` : Uygulama açılır açılmaz konsolun (Scanner) ekrana gelmesini sağlar.
- Burada kullanıcıdan bilgileri alıp bir `Siparis` nesnesi oluşturuyoruz ve onu `siparisService.siparisiKaydet(siparis)` diyerek yukarıdaki katmanlara gönderiyoruz.

SOLID

- * Modüllerin bağımlılığı iyi yönetilirse;
 - * tasarım daha kararlı olur, gerçeeklemedeki değışikliklerden (çok) etkilenmez
 - * "cohesion" artar "coupling" azalır.
 - * sonradan değışiklik kolaylaşır
 - * kod tekrar kullanımı/modülerlik artar
 - * yeni özellik kazandırma kolaylaşır
 - * anlaşılabilirlik artar
 - * karmaşıklık azalır (gereksiz bağımlılıklar ortadan kalkar)
 - * birim test yazımı kolaylaşır

SOLID

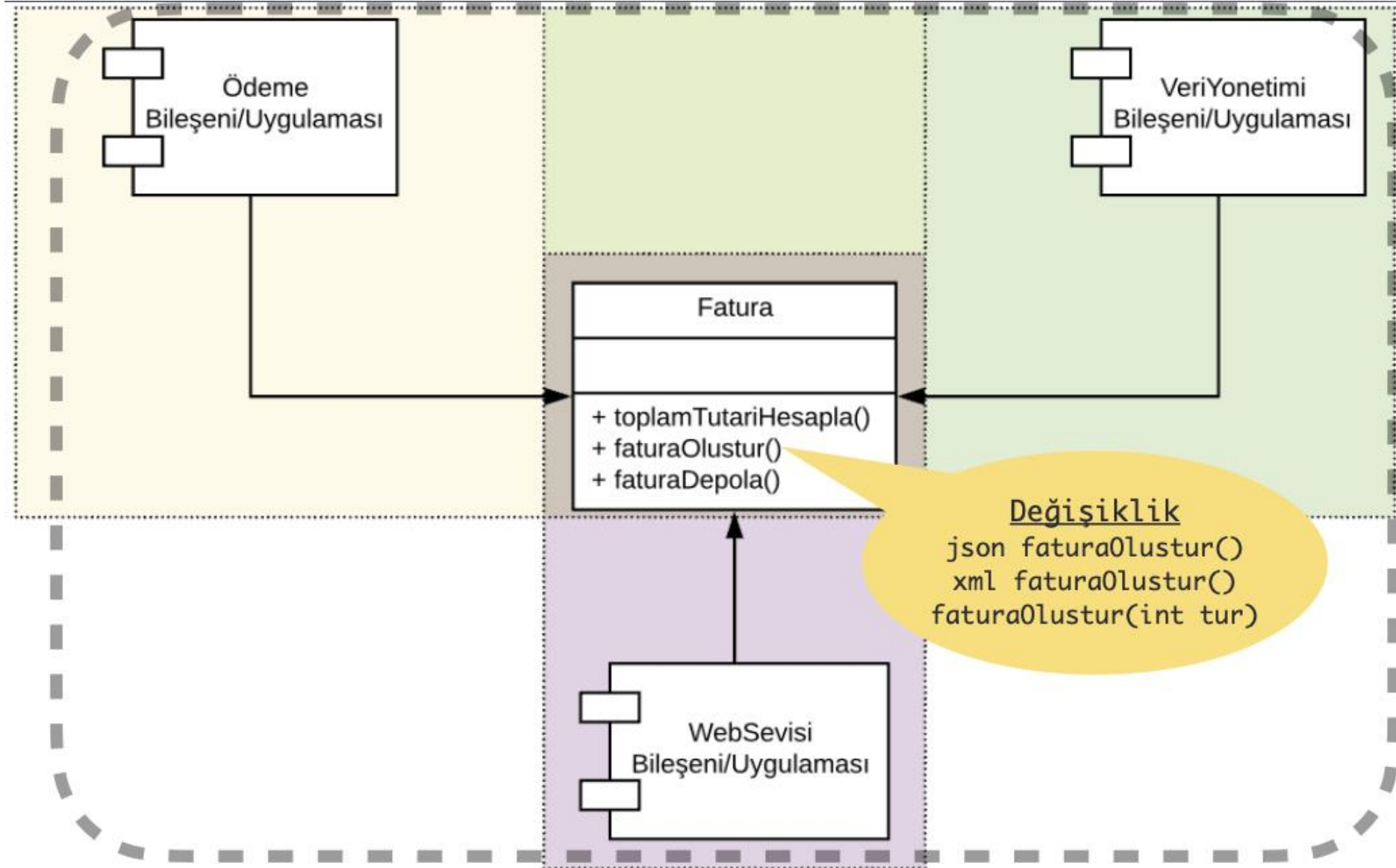
Kısaltma	Prensip Adı (Türkçe)	Temel Açıklama	Projedeki Karşılığı
SRP	Tek Sorumluluk Prensibi (Single Responsibility Principle)	Bir sınıfın değişmek için sadece bir nedeni olmalıdır.	Siparis sınıfı sadece veri tutar, SiparisRepository sadece DB işini yapar.
OCP	Açık/Kapalı Prensibi (Open Closed Principle)	Sınıflar gelişime açık, değişime kapalı olmalıdır.	Yeni bir raporlama özelliği eklemek için mevcut Siparis kodunu bozmak yerine yeni bir servis eklersin.
LSP	Liskov'un Yerine Geçme Prensibi (Liskov Substitution Principle)	Alt sınıflar, türetildikleri üst sınıfların yerine kullanılabilir.	JpaRepository'den türeyen senin repository'lerin, Spring'in tüm hazır metodlarını hatasız kullanır.
ISP	Arayüz Ayrımı Prensibi (Interface Segregation Principle)	Genel amaçlı tek bir arayüz yerine, özelleşmiş birden fazla arayüz kullanılmalıdır.	Tüm veritabanı işlerini tek bir dev interface'e değil; Urun ve Siparis için ayrı repository'lere böldük.
DIP	Bağımlılığın Tersine Çevrilmesi (Dependency Inversion Principle)	Sınıflar somut uygulamalara değil, soyutlamalara (arayüzlere) bağlı olmalıdır.	SiparisServisi somut bir sınıfa değil, ISiparisRepository arayüzüne bağlıdır (Dependency Injection).

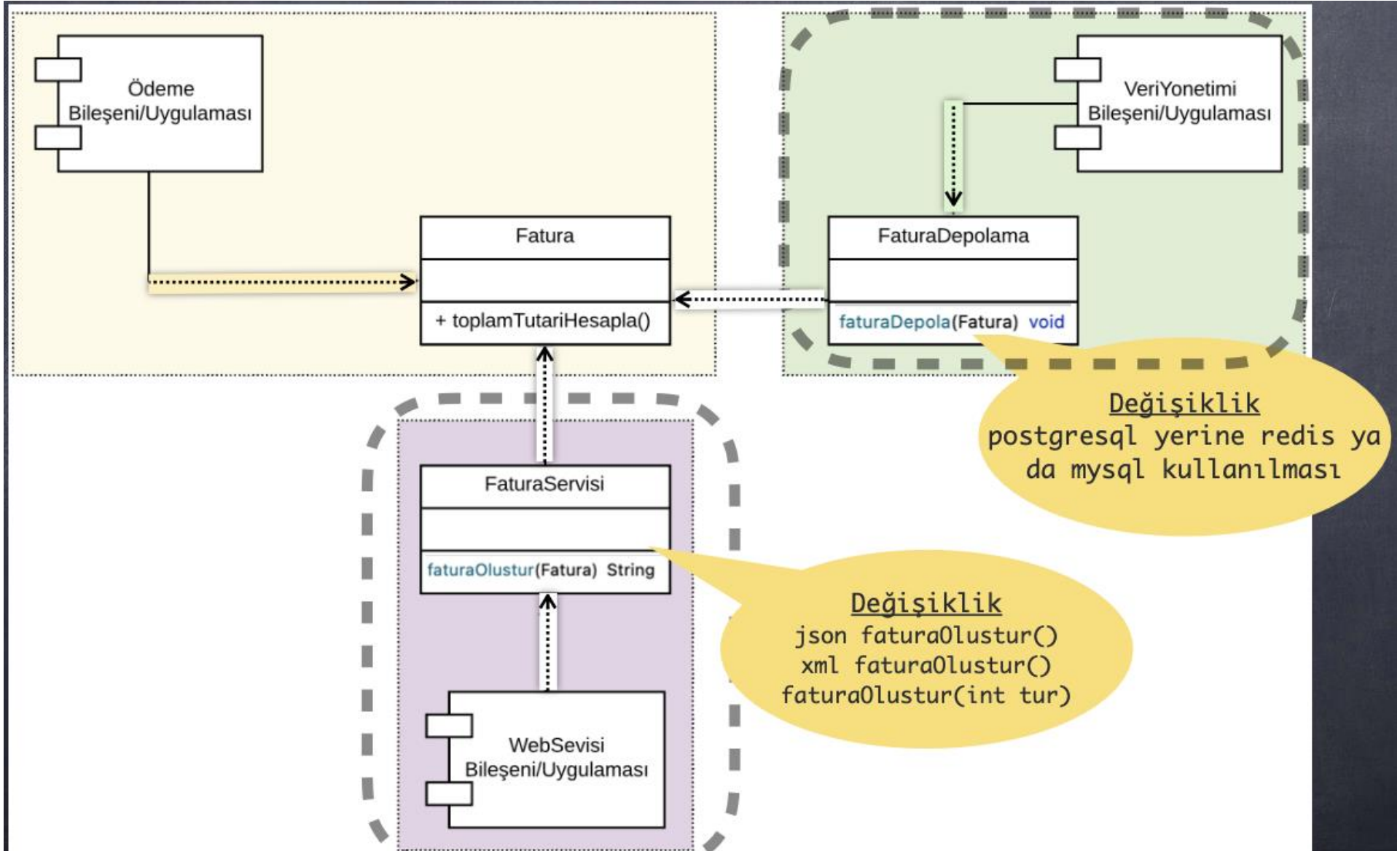
SRP

- Bir sınıf için; genel yönetim(business logic), veri yönetimi (saklama, erişme v.s.), sunum (gui, json, xml v.s.), iletim (gönder ,al v.s.), nesne oluşturma (factories), main vb. işlemlerinin her biri farklı sorumluluk olarak ele alınmalıdır.
- Bir sorumluluğu yerine getirirken çok sayıda nesne ve üye fonksiyon kullanabilir.

Bir modüle birden fazla sorumluluk yüklenirse;

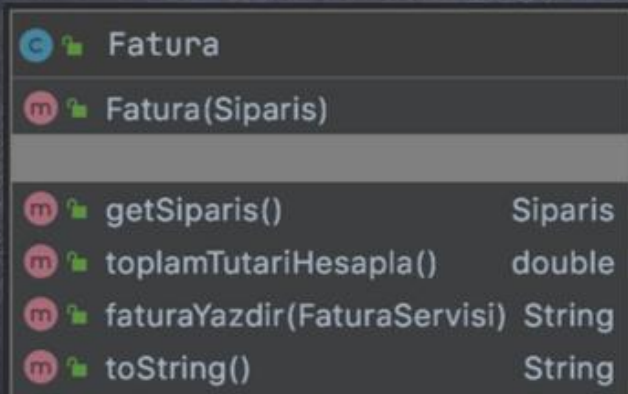
- değişiklik yapmak zorlaşır ve bakım aşamasının maliyeti yükselir
- değişikliğin yapıldığı modüle doğrudan bağlı modüllerde de değişiklik gerekebilir.
- doğrudan bağlı olmayan modüllerin tekrar derlenmesi, paketlenmesi, gönderilmesi v.s.
- gerekebilir
- sonunda her değişiklik çok karmaşık hale gelir ve yan etkiler ortaya çıkar (side effects, regresyon hataları)
- modülün tekrar kullanılabilme oranı azalır ve giderek projeye özel olmaya başlar (diğer projelerde, projenin diğer bölümlerinde kullanılabilme olasılığı azalır).
- anlaşılması ve yönetilmesi zorlaşır.
- birim testi yazmak zorlaşır



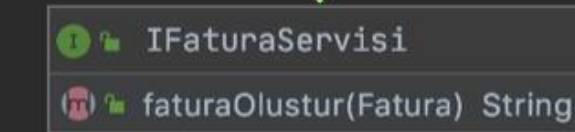
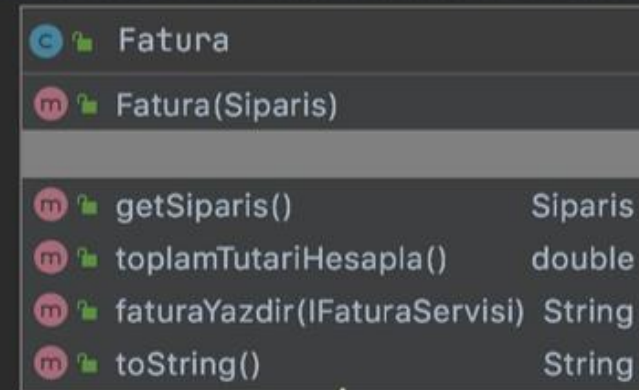


OCP

- Bu prensip, değişiklik yapmadan genişleyebilen modüller geliştirmeyi amaçlar.
- Mevcut kodu değiştirmeden yeni kodlar ekleyerek, yeni özellikler eklemeyi sağlar.
- “Open for extension”: geliştirilen modüller yeni gereksinimler ortaya çıktığında bunları desteklemek üzere genişleyebilmeli.
- “closed for modification”: geliştirilen modüller hata giderme dışında değiştirilmemeli.
- Mevcut kodda değişiklik yapmadan yeni özellik ekleyebilmenin yolu, bağlı olunan modüllerin gerçeklemeleri yerine soyutlamalarının kullanılmasıdır.



Değişiklik
faturaOlustur(Fatura, int tur)



LSP

- Barbara Liskov tarafından 1988 yılında ortaya konan bir ilkedir.
- Temel sınıf nesneleriyle çocukların nesneleri, istemci modül
- içerisinde sorun çıkarmadan yer değiştirebilmelidir.
- Kalıtım hiyerarşisinin tutarlı olarak belirlenmesini sağlar.
- Open/Closed ilkesine uymayan hiyerarşilerin önüne geçmemizi sağlar

AlanHesaplayici
alanHesapla(Dikdortgen) double



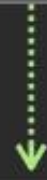
Dikdortgen
alanHesapla() double
yuksekluk double
genislik double



Kare
yuksekluk double
genislik double



AlanHesaplayici
alanHesapla(ISekil) double



ISekil
alanHesapla() double



Dikdortgen
alanHesapla() double
yuksekluk double
genislik double

Kare
alanHesapla() double
kenar double

